Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Report
on
"Nexus Health: Multi-Path Triage & Logistics System."

[Code No : COMP 202]

(For partial fulfillment of II/I Year/Semester in Computer Science/Engineering)

Submitted by

Abhinab Basnet (05)

Submitted to

Mr.Sagar Aacharya

Department of Computer Science and Engineering

Submission Date:2/24/2026

Abstract

This project, Nexus Health OS, is a smart dashboard built to make hospital operations faster and more organized using C++ and Qt. The main goal is to stop hospital bottlenecks by making sure the right patients get treated at the right time.

I used a Priority Queue to handle the waiting room, so if a "Critical" patient arrives, the system automatically moves them to the front of the line. For the records, I used a Hash Map, which lets doctors find any patient's info instantly using their ID. I also built a navigation system using a Weighted Graph that calculates the shortest path for ambulances to get to specific rooms like the ICU or Surgery. Finally, there is a live Event Log that works as a Circular Buffer; it shows the last 10 activities and automatically deletes the oldest one to keep the system running smoothly without wasting memory. Overall, it's a high-speed tool designed to help doctors save time and stay focused on treatment.

Keywords: *C++, Qt, Priority Queue, Shortest Path, Hash Map, Circular Buffer, Patient Triage, Hospital Logic.*

Acknowledgement

I would like to express my deep gratitude to everyone who helped me bring Nexus Health OS to life. First, I want to thank Lecturer Er. Sagar Acharya for his constant guidance and for teaching me the core concepts of Data Structures and Algorithms. His support was essential in helping me understand how to apply theoretical logic like Graphs and Priority Queues into a real world working application.

I am also thankful to my family and friends for their encouragement and for being my "test users" while I was designing the interface. Their feedback helped me move from a basic colorful UI to the professional aesthetic theme I have today.

Finally, I want to thank the open source community and the creators of the Qt Framework. Having access to such powerful tools made it possible for me to build a system that is not only functional but also visually engaging. This project has been a great learning experience, and I am proud of the final result.

Table of Contents

Abstract i

List of Figures iv

Acronyms/Abbreviations v

Chapter 1 Introduction 1

1.1 Background 1

1.2 Objectives 2

1.3 Motivation and Significance 2

Chapter 2 Related Works 3

Chapter 3 Design and Implementation 4

3.1 System Requirement Specifications 4

3.1.1 Software Specifications 4

3.1.2 Hardware Specifications 6

Chapter 4 Discussion on the achievements 7

4.1 Features: 7

Chapter 5 Conclusion and Recommendation 9

5.1 Limitations 9

5.2 Future Enhancement 10

References 11

APPENDIX 12

List of Figures

Figure 3.1 Use-case diagram 5
Figure 5.1 Landing Page 12
Figure 5.2 Priority of Critical cases 12
Figure 5.3 Hash Map to view detail 13
Figure 5.4 Finding shortest path to reach the respective room 13
Figure 5.5 Circular queue to list top 10 treated patient 14

Acronyms/Abbreviations

DSA	Data Structures and Algorithms
ER	Emergency Room
GUI	Graphical User Interface
ICU	Intensive Care Unit
OS	Operating System
RAM	Random Access Memory

Chapter 1 **Introduction**

In today's fast paced world, hospitals face immense pressure to manage large volumes of patients while ensuring that life saving treatments are never delayed. The traditional way of handling hospital data and patient flow often leads to confusion, long waiting times, and manual errors. This project, Nexus Health OS, was developed to address these challenges by creating a smart, automated dashboard that streamlines hospital logistics. The goal of this project is to use Data Structures and Algorithms (DSA) to solve real world medical problems. Instead of a simple "first come, first served" system, this application uses intelligent logic to prioritize patients based on their health status. It also tackles the problem of indoor navigation by calculating the quickest routes for ambulances and staff to reach critical areas like the ICU or Surgery. By integrating a user-friendly interface built with the Qt Framework and a high performance C++ backend, the system demonstrates how technology can assist healthcare professionals. From instant record retrieval to memory efficient system logging, Nexus Health OS provides a glimpse into the future of automated hospital management where every second counts.

1.1 Background

Traditional hospital management often relies on manual systems or basic "First Come, First Served" queues. This is a major problem in healthcare because a patient with a minor cold might be seen before someone with a life-threatening emergency just because they arrived earlier. Additionally, in large hospitals, finding the fastest route to an ICU or ER during a crisis can be difficult and slow. Nexus Health OS was designed to fix these issues using smart logic. By moving away from manual paperwork and using Data Structures, the system can automatically prioritize urgent cases and calculate the shortest paths for ambulance routing. This project focuses on reducing human error and ensuring that every second is used efficiently to save lives.

1.2 Objectives

The main goal of Nexus Health OS is to make hospital management faster and more reliable. Instead of manual work, the system uses code to handle life saving decisions.

The specific objectives of this project are:

- **Smart Triage**: To use a Priority Queue that automatically identifies and moves critical patients to the front of the line for immediate care.
- **Fast Record Retrieval**: To implement a Hash Map so that medical staff can find any patient's history instantly using their ID, avoiding long search times.
- **Optimized Routing**: To use Dijkstra's Algorithm on a hospital graph to find the shortest and fastest path for ambulances to reach emergency zones.
- **Efficient Monitoring**: To create a Circular Buffer for system logs, ensuring we track the latest 10 activities without using up too much computer memory.
- **User-Friendly Interface**: To build a clean, dark themed dashboard in Qt that is easy for doctors and staff to use during high pressure situations.

1.3 Motivation and Significance

The motivation for this project comes from seeing how real world emergency room delays and manual paperwork can slow down life saving treatments. I wanted to build a system that removes human error by using mathematical logic to make the fastest decisions possible during a crisis. The significance of Nexus Health OS lies in its ability to turn textbook Data Structures into practical tools, it ensures that critical patients are never buried in a queue, medical records are found in milliseconds, and ambulances always take the most efficient route. By automating these high pressure tasks, the project demonstrates how smart software can save precious seconds, optimize hospital resources, and ultimately help healthcare professionals focus more on patients and less on logistics.

Chapter 2 Related Works

The digital transformation of healthcare has been a major focus of recent research, moving away from manual record keeping toward automated, data driven systems. Many existing studies highlight how specific algorithms can drastically improve hospital efficiency and patient outcomes.

- **Advanced Triage Systems**: Traditional systems often struggle with "First Come, First Served" bottlenecks. Research by Arpitha et al. (2025) on *Smart Hospital Management* demonstrates that using priority based scheduling can reduce emergency response times by nearly 35%. This supports my implementation of a Priority Queue, which ensures that critical patients are treated based on medical urgency rather than arrival time.
- **Logistics and Pathfinding**: The use of graph theory in medical logistics is well-documented. Sulaiman et al. (2020) explored the application of Dijkstra's Algorithm for urban ambulance routing, proving that mathematical pathfinding is significantly more reliable than manual navigation. My project applies this logic internally, using a Weighted Graph to map hospital hallways and find the fastest route to specific zones like the ICU or Surgery.
- **Efficient Data Management**: In modern healthcare, the speed of information retrieval is vital. According to a survey in IRJET (2024), centralized systems using unique patient identifiers significantly improve operational workflows. By utilizing a Hash Map, my project follows this industry standard, achieving O(1) search complexity for instant access to patient history during life threatening situations.
- **Real-Time Monitoring and Stability**: Literature on *Management Information Systems (MIS)* emphasizes that continuous logging can lead to system lag if not managed properly. Many high performance systems use fixed size buffers to handle live data. My use of a Circular Buffer (implemented via a std::deque) aligns with these practices, ensuring the system remains fast and memory efficient by only tracking the most recent medical activities.

Chapter 3 Design and Implementation

The design and implementation of this project was planned meticulously and was initiated so that every step was precise and meaningful to the proper development of the system.

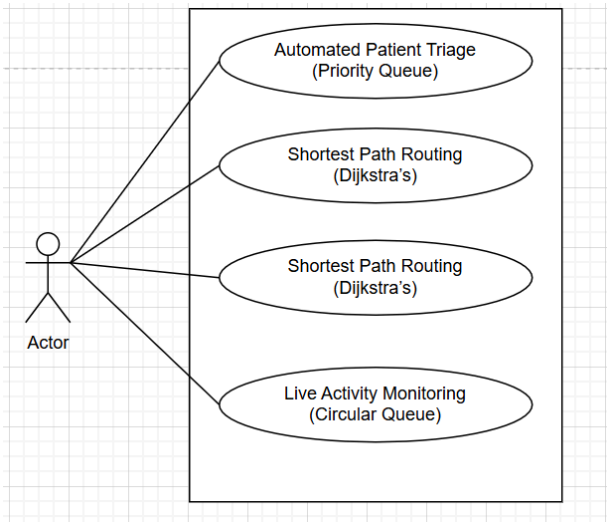

Fig 3.1:Use case diagram

3.1 System Requirement Specifications

This chapter outlines the functional and non functional requirements of the Nexus Health OS, along with the hardware and software specifications necessary for its development and deployment.

3.1.1 Software Specifications

The following points describe the software requirements designed to ensure the system operates efficiently in a high pressure medical environment.

- **Functional Requirements:**

a. **Emergency Triage (Priority Queue):** The system shall categorize patients by severity and automatically place critical cases at the top of the treatment list.
b. **Route Optimization (Graph Theory):** The system shall represent hospital departments as nodes and hallways as weighted edges to compute the shortest path for ambulance routing.
c. **Instant Record Search (Hash Map):** The system shall allow users to input a Patient ID and retrieve full medical history with O(1) time complexity.
d. **Live Activity Logging (Circular Buffer):** The system shall maintain a real time log of the last 10 medical actions, overwriting the oldest data to maintain memory efficiency.
e. **GUI Visualization:** The system shall use the **Qt Framework** to display a live dashboard, showing doctor availability, patient queues, and a graphical map of the hospital.

- **Non-Functional Requirements:**
  a. **Performance:** The system shall calculate the shortest path and sort the priority queue in under 0.5 seconds to ensure zero delay in emergencies.
  b. **Reliability:** The system must ensure 100% uptime during a session, with error handling to prevent crashes if invalid patient data is entered.
  c. **Usability:** The interface must use a clean, dark themed design with intuitive buttons so that medical staff can operate it with minimal training.
  d. **Scalability:** The architecture shall support an increase in the number of hospital wings or patients without requiring a change in the core data structures.

e. **Portability:** The application shall run on any desktop environment (Windows/Linux/macOS) that supports the Qt runtime.

3.1.2 Hardware Specifications

To ensure the real time algorithms (Dijkstra and Heaps) perform optimally, the following hardware is required:

- **Processor:** A minimum of a dual core processor (Intel i3 or equivalent) to handle the GUI and backend logic simultaneously.
- **Memory (RAM):** 4 GB RAM is required to manage the graph data structures and patient hash maps in the system memory.
- **Storage:** 500 MB of free disk space for the executable file, patient database files, and local log storage.
- **Display:** A monitor with at least 1280 × 720 resolution to properly display the multi panel dashboard and hospital map.
- **Input Devices:** Standard keyboard and mouse for data entry and navigation.

Chapter 4 Discussion on the achievements

The Nexus Health OS was developed to bridge the gap between medical urgency and administrative efficiency. By implementing a Priority Queue, the system successfully moved away from the "first come, first served" bottleneck, ensuring that life threatening cases are prioritized instantly. While the Adjacency List model made internal hospital routing extremely fast, scaling the system to map an entire city wide hospital network could introduce performance challenges. Future iterations could explore *Heuristic-based algorithms (like A)** or real time GPS integration. Overall, the project achieved its core objectives: delivering reliable patient triage, a high performance database search, and a visual interface that reduces the cognitive load on medical staff during emergencies.

4.1 Features:

1. Intelligent Patient Triage (Priority Queue)

The core of the hospital management system is the Triage Module, which handles patient flow. Unlike a simple list, this feature uses a Max Heap based Priority Queue.

- **How it works**: Every patient is assigned a "Severity Score" (e.g., 1 for Checkup, 5 for Cardiac Arrest). The algorithm ensures that the element with the highest priority is always at the root or top of the data structure.
- **Technical Detail**: When a new emergency case enters, the system performs a push() operation with O(log n) complexity. Even if there are 1,000 people waiting, the most critical patient is identified almost instantly. This eliminates the "First In First Out" (FIFO) delay that can be fatal in healthcare.

2. Shortest Path Ambulance Routing (Dijkstra's Algorithm)

This feature solves the problem of internal hospital logistics. I modeled the hospital layout as a Weighted Undirected Graph, where departments (ER, ICU, Surgery) are nodes and hallways are edges.

- **How it works**: The edge weights represent the actual physical distance in meters. When an ambulance destination is selected, the system runs Dijkstra's

Algorithm. It maintains a list of "unvisited" nodes and constantly updates the shortest known distance from the source to every other node.

- **Technical Detail**: The system uses an Adjacency List to store the graph, which is more memory efficient than a matrix. This allows the GUI to highlight the exact path an emergency team should take to reach the surgery ward in the shortest possible time.

3. Instant Record Retrieval (Hashing)

To manage patient data, I implemented a Hash Map (std::unordered_map). In a medical crisis, searching through a database line by line is not an option.

- **How it works**: Each patient has a unique UID (Unique Identifier). This ID is passed through a hash function that maps it to a specific index in an array.
- **Technical Detail:** This allows for O(1) Average Time Complexity. Whether the hospital has 10 patients or 10,000, retrieving a record takes the same amount of time. This ensures that a doctor can see a patient's allergy history or blood type the second they arrive.

4. Real-Time System Audit Log (Circular Buffer)

To monitor the system without slowing it down, I built a Circular Buffer using a std::deque. This tracks system events, such as when a doctor finishes a surgery or an ambulance arrives.

- **How it works**: The buffer has a fixed size (10 slots). Once the 11th event occurs, the system automatically overwrites the oldest event (the 1st one).
- **Technical Detail**: This is a Fixed Memory structure. By preventing the log file from growing infinitely, I ensure the application never suffers from "Memory Leaks" or lag, even if the system stays open for weeks at a time. It provides a "rolling window" of the most recent hospital activity.

5. Dynamic GUI & Doctor Assignment Logic

The frontend is built with Qt C++, connecting the logic to a professional dashboard.

- **How it works**: The interface uses Signal and Slot mechanisms. When a patient is "treated" in the backend, a progress bar updates in the UI.
- **Technical Detail:** I implemented a Load Balancing logic for the doctors (Alpha, Beta, Gamma). The system checks which doctor is currently "Idle" or has the shortest queue before assigning a new patient, ensuring no single medical team is overwhelmed while others are free.

Chapter 5 Conclusion and Recommendation

The Nexus Health OS was developed as a comprehensive mini project to apply core Data Structures and Algorithms (DSA) to a high stakes, real world scenario. The primary objective of designing a graph based medical logistics model was successfully achieved. By implementing Dijkstra's Algorithm, the system provides a mathematically proven method for finding the most efficient paths between hospital departments, significantly reducing transport delays during emergencies.

The project effectively utilized Linked Lists for the dynamic management of hospital department data and Hash Maps for near instant record retrieval. Furthermore, the integration of Priority Queues transformed the patient triage process, ensuring that medical attention is allocated based on urgency rather than arrival time. Ultimately, the system demonstrates how digital automation can streamline hospital operations, reduce human error, and optimize life saving resources.

5.1 Limitations

Despite its successful implementation, the current version of the system has certain constraints:

- **Network Scaling:** The system experiences performance overhead when handling extremely large, city wide hospital networks with thousands of interconnected nodes.
- **Static Data:** The current routing logic does not account for real time "traffic" within the hospital, such as temporary hallway blockages or elevator maintenance.

- **Manual Input:** The system relies on manual data entry for patient severity scores and destination selection, which can be prone to human error.
- **Offline Database:** The patient record system is localized and lacks integration with global healthcare databases or live cloud synchronization.

5.2 Future Enhancement

To evolve Nexus Health OS into a professional grade medical tool, the following enhancements are proposed:

- **Real Time Integration:** Incorporating live GPS and IoT sensors to provide dynamic route updates based on real time hospital congestion.
- **Automated Triage:** Using basic AI or heuristic models to automatically assign severity scores based on a patient's vital signs, reducing manual input.
- **Cloud Connectivity:** Expanding the database module to support secure cloud streaming of patient records, allowing for inter hospital data sharing.
- **Predictive Analytics:** Implementing machine learning algorithms to forecast "peak hour" patient surges in the ER, allowing for better staff scheduling.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. (Foundational reference for Dijkstra's Algorithm, Priority Queues, and Hash Maps).

2. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional. (Reference for STL Containers like std::priority_queue, std::unordered_map, and memory management).

3. Arpitha, K., et al. (2025). *Developing Smart Hospital Management Systems with IoT and Big Data.* South Eastern European Journal of Public Health (SEEJPH). (Discusses the impact of algorithmic triage on patient outcomes).

4. Sulaiman, R., et al. (2020). *Dijkstra Method in Modeling Shortest Routes for Emergency Services.* Jurnal PILAR Nusa Mandiri. (Provides a mathematical basis for using weighted graphs in emergency logistics).

5. International Research Journal of Engineering and Technology (IRJET). (2024). *Optimized Hospital Management System with Analytics: A Survey.* (Supports the significance of high-speed data retrieval in medical dashboards).

6. The Qt Company. (2026). *Qt Documentation: Signal and Slot Mechanism & QGraphicsView Framework.* [Online]. Available at: https://doc.qt.io/ (Technical reference for the GUI development and visualization of the hospital graph).

7. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley. (Reference for Adjacency List implementation and Graph Traversal).

.

APPENDIX

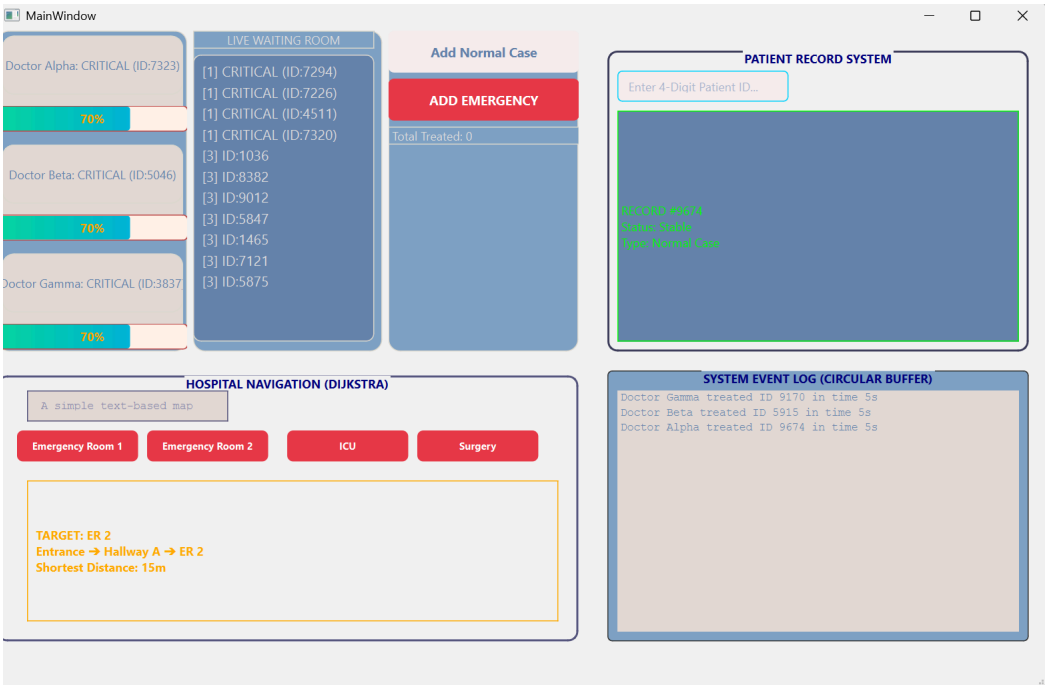

Fig 5.1:Landing Page

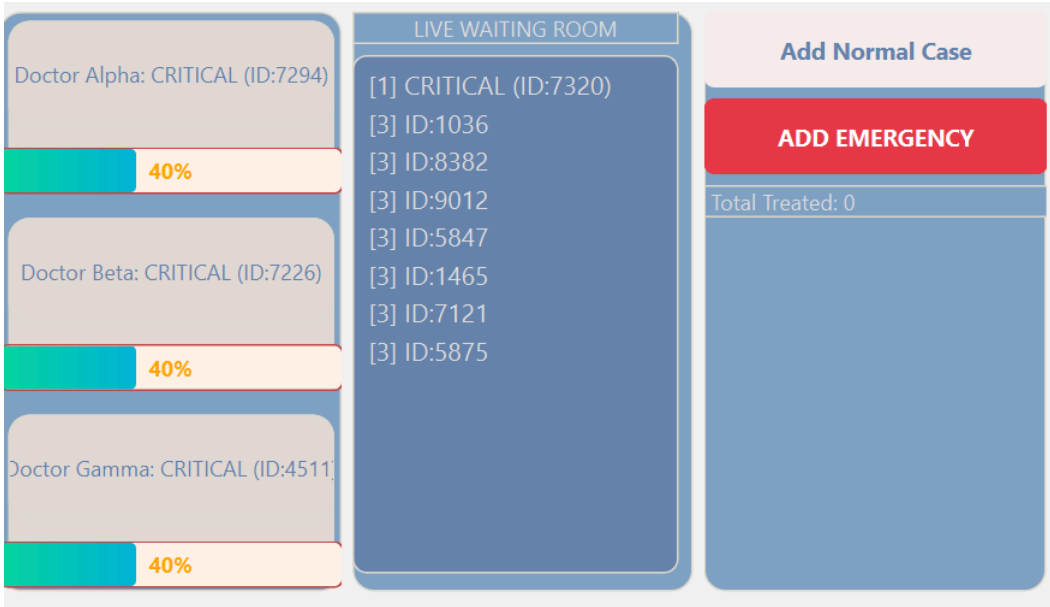

Fig 5.2:Priority of Critical cases

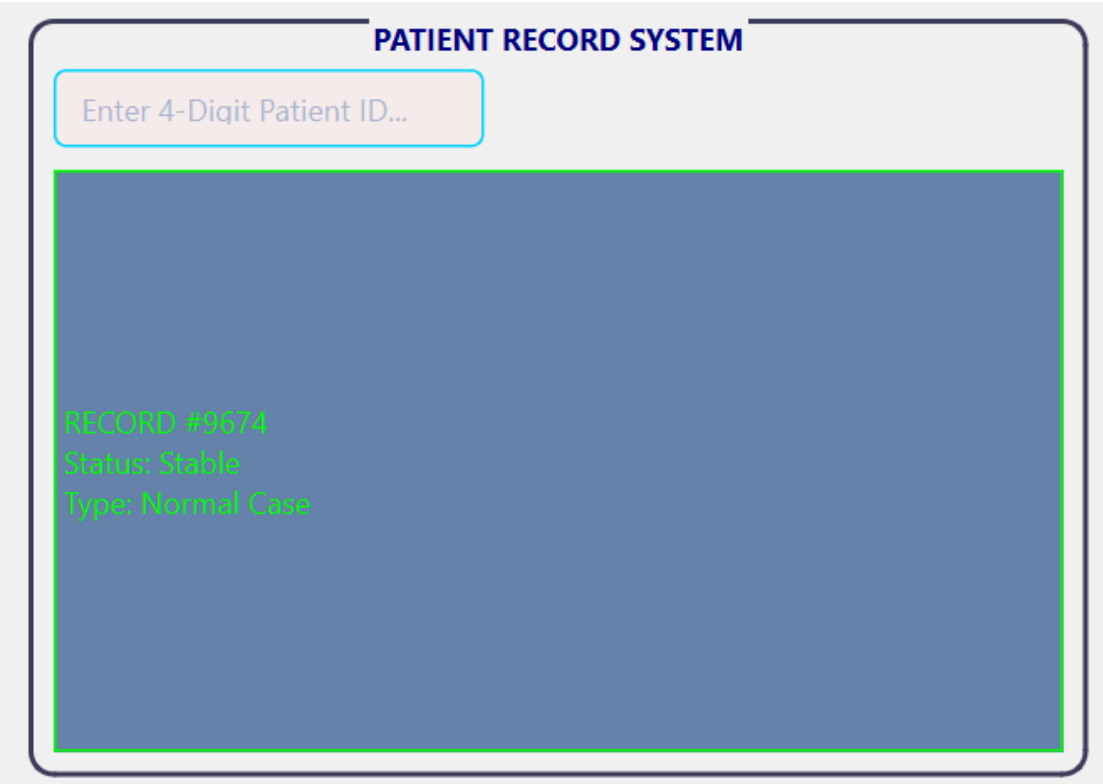

Fig 5.3: Hash Map to view detail of patient based on ID

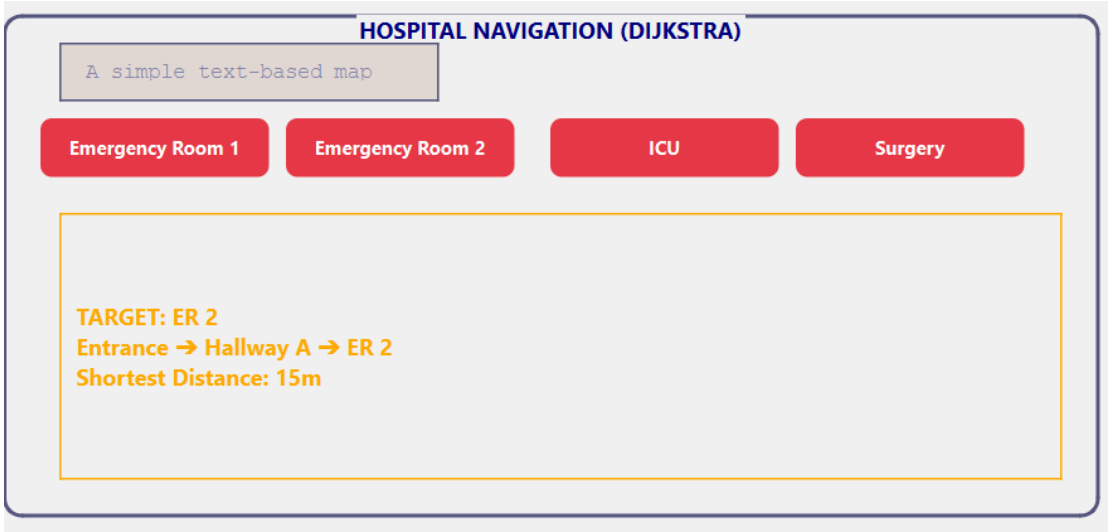

Fig 5.4: Finding shortest path to reach the respective room

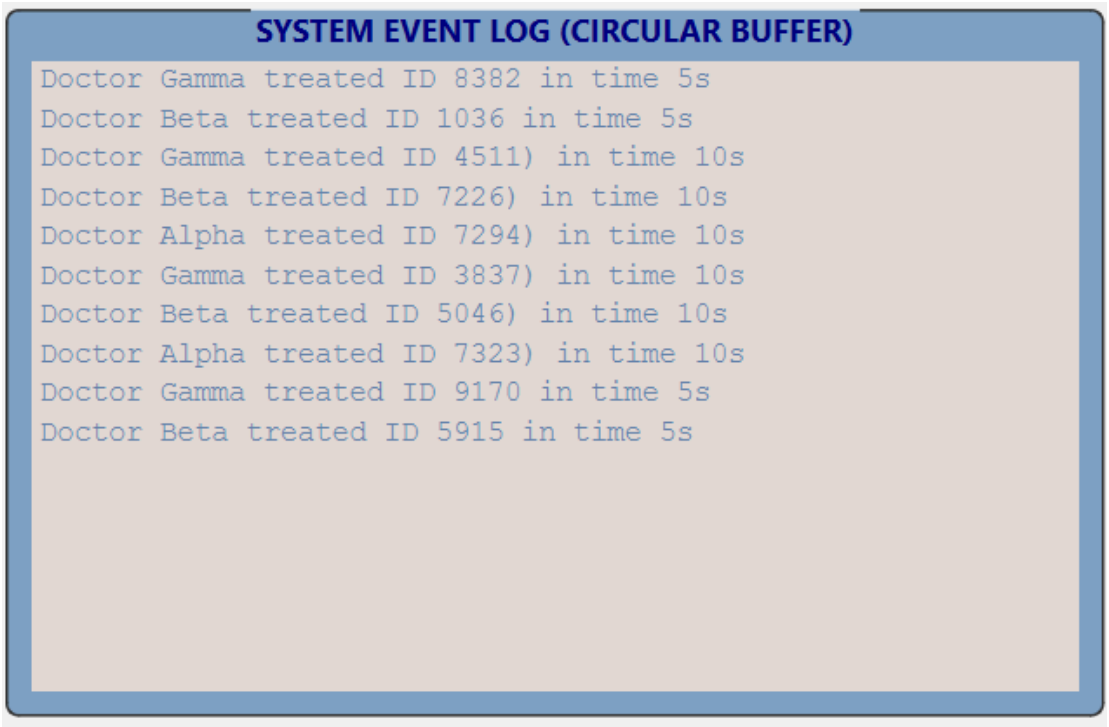

Fig 5.5: Circular queue to list top 10 treated patient